```
    (planar, cache) ← isPlanar(subG,
cache)                          #5
    if !planar then
      cache[graph] ← false
      return (false, cache)
  for e in graph.edges
do                                              #6
    subG ← graph.remove(e)
    (planar, cache) ← isPlanar(subG, cache)
    if !planar then
      cache[graph] ← false
      return (false, cache)
  cache[graph] ←
true                                                #7
  return (true, cache)
```

#1 First, we check if the graph is in out cache – the cache can be as simple as a dictionary, as long as graphs are serializable.

#2 If the number of edges and vertices violates Euler's constraints for planar graphs, we know the graph is not planar: before returning, we need to update the cache so this graph won't be checked again in another branch of the computation.

#3 Instead of looking for K_5 or $K_{3,3}$ only, we just check if it is complete (since graph has at least 5 vertices, then it's for sure non-planar) or complete bipartite (in this case, we also need to check that the smallest of the two partitions has size 3).

#4 Cycle through the graph's vertices.

#5 When we perform a recursive call, we need to update the cache as well... Besides the cache, the algorithm stays the same.